

SecurAx

Auteur	Abdallah Benaicha
Nature	Projet de Fin d'Études (PFE)
Domaine	Cybersécurité Offensive & Défensive
Version	3.0 — Édition Haute Contraste & Complète
Date	Juin 2026

1. INTRODUCTION ET CONTEXTE GÉNÉRAL

1.1 Contexte et Motivations Fondamentales

Dans l'écosystème numérique actuel, la sécurité applicative n'est plus un luxe réservé aux grandes entreprises technologiques, mais une nécessité vitale pour toute organisation. Cependant, la complexité des outils de sécurité professionnels (souvent en ligne de commande ou avec des interfaces austères) et leur coût prohibitif créent une barrière à l'entrée significative pour les PME et les développeurs indépendants.

"La sécurité doit fonctionner pour de vraies personnes, pas seulement pour des experts. Si un outil nécessite un doctorat pour être compris, il échoue dans sa mission."

— Dan Kaminsky, Chercheur en Cybersécurité

SecurAx a été conçu précisément pour combler ce fossé. Il s'agit d'une plateforme unifiée d'intelligence de sécurité (Security Intelligence Platform) qui démocratise l'audit de vulnérabilités sans sacrifier la profondeur technique. Contrairement aux scanners automatisés traditionnels qui génèrent des milliers de faux positifs, SecurAx intègre un moteur de corrélation intelligent (ARIA) pour contextualiser les menaces.

Problème de l'industrie	Solution apportée par SecurAx	Impact
Outils fragmentés (Nmap, ZAP, Bandit)	Interface unique orchestrant 8+ moteurs sous-jacents	Gain de temps massif
Résultats bruts incompréhensibles	Agent IA (ARIA) traduisant le risque technique en impact business	Décisions accélérées
Faux positifs abondants	Moteur de risque v3 avec détection de chaînes d'attaque réelles	Réduction du bruit (alert fatigue)
Barrière linguistique	Support natif complet Français / Anglais / Arabe	Accessibilité accrue (MENA)
Rapports statiques	Génération dynamique de PDF encyclopédiques avec code de remédiation	Actionnabilité immédiate

✓ **BONNE PRATIQUE** : L'approche SecurAx permet à un développeur de réaliser en 15 minutes un audit de base qui prendrait normalement 2 jours à un consultant.

1.2 Genèse du Projet et Décisions Architecturales

Ce projet de fin d'études a débuté avec une question simple : *Comment construire un scanner de vulnérabilités qui soit à la fois puissant sur le plan offensif et défendable de par sa propre architecture ?*

1	Backend Python/Flask	L'écosystème Python domine la sécurité (Scapy, Impacket, Nmap). Flask offre la légèreté nécessaire contrairement à Django, tout en permettant une structure modulaire stricte via les Blueprints.
2	Frontend React/Vite	Nécessité d'une interface très réactive et moderne (SPA) pour gérer les dashboards en temps réel et la topologie réseau, impossible avec de simples templates Jinja2.
3	Base de données hybride	SQLite/WAL mode pour le développement local et les déploiements on-premise isolés, avec abstraction via SQLAlchemy pour basculer sur PostgreSQL en production SaaS.
4	Séparation des privilèges	L'exécution des scans actifs (DAST) est isolée des processus d'analyse statique (SAST), prévenant les fuites de mémoire et les blocages du thread principal WSGI.

1.3 Périmètre Fonctionnel Complet

SecurAx couvre trois piliers fondamentaux de la sécurité offensive et défensive :

Pilier	Fonctionnalités	Cible
Offensif (Red Team)	Scan web actif, DAST (ZAP API), Nmap exhaustif, Fuzzing de paramètres	Applications en cours d'exécution
Défensif (Blue Team)	Analyse statique (Bandit/Semgrep), Audit de configuration Apache/Nginx, Scan dépendances	Code source et infrastructure
Intelligence (Purple Team)	Moteur de risque CVSS v3.1, Génération de chaînes d'attaque, Assistant LLM (ARIA)	Équipes SOC et décideurs

2. ARCHITECTURE GÉNÉRALE ET DESIGN PATTERNS

2.1 Vue d'Ensemble — Layered Architecture

SecurAx implémente une architecture en couches (Layered Architecture) stricte. Cette séparation des préoccupations garantit que les erreurs dans une couche ne compromettent pas l'ensemble du système, principe clé de la *Defense in Depth*.

Couche	Responsabilité technique	Fichiers / Modules clés	Contrôles de sécurité associés
Présentation (React)	Rendu UI, validation client, SPA routing	frontend/src/components/*	CORS, CSP, XSS Escaping (React DOM)
API Gateway (Flask Blueprints)	Routage HTTP, Auth, Rate Limiting	backend/blueprints/*.py	JWT/Session validation, CSRF tokens, Limiters
Business Logic (Services)	Orchestration des scans, Moteur IA	scanners/*, ai_agent.py, risk_engine.py	Input sanitization, Timeout controls, Sandboxing
Data Access (SQLAlchemy)	Abstraction BDD, requêtes ORM	backend/database.py, models.py	Requêtes paramétrées (anti-SQLi), Chiffrement (bcrypt)
Infrastructure (OS/Network)	Exécution binaire, réseau	nmap, bandit, système hôte	Least privilege (utilisateur non-root), Chroot jail

2.2 Application Factory Pattern

Flask recommande le pattern 'Application Factory' pour les projets d'envergure. Plutôt que de créer l'instance Flask au niveau global du module (`app = Flask(__name__)`), SecurAx l'instancie dans une fonction `create_app()`. Cela permet d'isoler les environnements de test et de production, et prévient les imports circulaires massifs.

```
# backend/app.py - Implémentation de l'Application Factory
from flask import Flask
from .extensions import db, limiter, csrf, login_manager
from .middleware import SecurityHeadersMiddleware

def create_app(config_object='backend.config.ProductionConfig'):
    app = Flask(__name__)
    app.config.from_object(config_object)

    # 1. Initialisation des extensions sécurisées
```

```

db.init_app(app)
csrf.init_app(app)
limiter.init_app(app)
login_manager.init_app(app)

# 2. Ajout du Middleware de sécurité natif (headers HTTP purs)
app.wsgi_app = SecurityHeadersMiddleware(app.wsgi_app)

# 3. Enregistrement des Blueprints (Routage modulaire)
from .blueprints.auth import auth_bp
from .blueprints.scans import scans_bp
app.register_blueprint(auth_bp, url_prefix='/api/auth')
app.register_blueprint(scans_bp, url_prefix='/api/scans')

return app

```

■ Avantage sécurité

Si l'application est compromise via une variable globale, l'attaquant gagne accès à tout le contexte. L'Application Factory force l'état à être confiné à l'instance locale exécutée par le worker Gunicorn.

2.3 Système RBAC (Role-Based Access Control)

SecurAx implémente un contrôle d'accès basé sur les rôles extrêmement strict, s'attaquant directement au risque OWASP A01:2021 (Broken Access Control). Un décorateur personnalisé `require\_permission` intercepte la requête avant l'exécution de la vue.

```

# backend/models.py - Décorateur d'autorisation RBAC
from functools import wraps
from flask_login import current_user
from flask import abort

def require_permission(permission_name):
    def decorator(f):
        @wraps(f)
        def decorated_function(*args, **kwargs):
            if not current_user.is_authenticated:
                abort(401) # Unauthorized

            # Vérification dans la hiérarchie des rôles
            if not current_user.has_permission(permission_name):
                # Log de l'incident de sécurité (Audit Trail)
                log_security_event(
                    user_id=current_user.id,
                    event_type='ACCESS_DENIED',
                    resource=permission_name
                )
        return decorated_function
    return decorator

```

```
        abort(403) # Forbidden

    return f(*args, **kwargs)
    return decorated_function
    return decorator

# Utilisation dans un Blueprint :
@admin_bp.route('/users/delete/<int:id>', methods=['POST'])
@require_permission('manage_users')
def delete_user(id):
    # Seuls les admins atteindront cette ligne
    pass
```

2.4 Middleware de Sécurité Global

Plutôt que de dépendre de Flask pour injecter les headers de sécurité, SecurAx utilise un middleware WSGI. Cela garantit que même si Flask plante (renvoyant une erreur 500 par défaut générée par Werkzeug), les headers de sécurité stricts (HSTS, CSP, etc.) seront **toujours** appliqués à la réponse HTTP brute.

Header injecté	Valeur	Protection ciblée
Strict-Transport-Security	max-age=31536000; includeSubDomains	Force HTTPS (contre MITM / SSL Stripping)
Content-Security-Policy	default-src 'self'; script-src 'self'	Cross-Site Scripting (XSS), Data injection
X-Content-Type-Options	nosniff	MIME-sniffing bypass (exécute un PNG comme JS)
X-Frame-Options	DENY	Clickjacking (UI redressing)
Permissions-Policy	geolocation=(), microphone=()	Restriction d'accès au matériel client
Server	Hidden / SecurAx	Information Disclosure (CWE-200)

3. TECHNOLOGIES ET BIBLIOTHÈQUES

PYTHON — ANALYSE ENCYCLOPÉDIQUE

Ce chapitre dissèque l'écosystème technique sur lequel repose SecurAx. Pour chaque bibliothèque majeure, nous analysons son rôle, son fonctionnement interne, ses alternatives et la manière dont elle est intégrée de façon sécurisée.

3.1 Flask 3.1.2 — Micro-Framework de Référence

Flask, créé en 2010 par Armin Ronacher, est le standard de facto pour la création d'APIs en Python. Basé sur le toolkit WSGI Werkzeug et le moteur de templates Jinja2, sa philosophie 'micro' signifie qu'il ne force ni ORM, ni structure de projet.

Critère	Flask (SecurAx)	Django	FastAPI
Philosophie	Micro, modulaire, explicite	Batteries-included, monolithique	Moderne, asynchrone, types stricts
Routing	Décorateurs @app.route	Fichier urls.py centralisé	Décorateurs @app.get
Sécurité native	Minime (Jinja autoescape)	Maximale (CSRF, Auth, Clickjack)	Dépendante du développeur
Performance	Synchrone (WSGI), moyennement rapide	Lourd, lent au démarrage	Asynchrone (ASGI), très rapide
Choix pour SecurAx	Contrôle total sur l'architecture requise pour orchestrer Nmap/ZAP	Trop rigide pour des outils système	Inutile (les scanners sont I/O bloquants de toute façon)

■ **ATTENTION** : Le serveur de développement intégré de Flask (app.run()) NE DOIT JAMAIS être utilisé en production. Il n'est pas conçu pour être performant, sécurisé, ni pour gérer des requêtes simultanées massives. SecurAx utilise Gunicorn pour la production.

3.2 bcrypt 5.0.0 — Protection des Credentials

Développé par Niels Provos et David Mazières en 1999, bcrypt est une fonction de hachage de mots de passe basée sur l'algorithme de chiffrement symétrique Blowfish. Sa caractéristique principale est sa lenteur ajustable via le `work\_factor` (cost).

```
# backend/models.py - Hachage robuste avec Bcrypt
import bcrypt
```

```
def set_password(self, plain_text_password):
    # 1. Encodage strict en bytes
    password_bytes = plain_text_password.encode('utf-8')

    # 2. Génération d'un sel cryptographique fort (16 bytes)
    # bcrypt.gensalt() génère un sel unique pour CHAQUE utilisateur
    # rounds=12 demande ~400ms de calcul (protection anti-bruteforce)
    salt = bcrypt.gensalt(rounds=12)

    # 3. Hachage = Algorithme + Cost + Sel + Mot de passe
    hashed = bcrypt.hashpw(password_bytes, salt)

    # 4. Stockage du hash final en base (contient le sel et le cost)
    # Format : $2b$12$Salt(22chars)Hash(31chars)
    self.password_hash = hashed.decode('utf-8')
```

■ Protection Rainbow Tables

Les 'Rainbow Tables' sont des bases de données de milliards de mots de passe pré-hachés. Parce que bcrypt inclut un sel généré aléatoirement *pour chaque mot de passe*, le même mot de passe 'admin123' aura deux hashes totalement différents pour deux utilisateurs. Cela rend les attaques par Rainbow Tables mathématiquement inutiles.

3.3 PyOTP 2.9.0 — Implémentation MFA RFC 6238

L'authentification multifacteur (MFA) est indispensable. SecurAx évite les SMS (vulnérables au SIM Swapping) et utilise TOTP (Time-based One-Time Password), le protocole standard utilisé par Google Authenticator et Authy.

1	Génération Secret (Serveur)	PyOTP génère une chaîne Base32 de 32 caractères (ex: JBSWY3DPEHPK3PXP) unique à l'utilisateur. Cette clé ne transite qu'une seule fois via QR Code.
2	Synchronisation (Client)	L'app Authenticator scanne le QR. Le secret ne quitte plus le téléphone de l'utilisateur (Cold storage effect).
3	Génération Code (Client)	Toutes les 30s, l'app calcule : HMAC-SHA1(Secret, Timestamp_Actuel) et extrait un code à 6 chiffres.
4	Validation (Serveur)	PyOTP effectue le même calcul avec l'heure du serveur. Si les codes correspondent, l'accès est accordé.

3.4 Bandit 1.8.3 — Analyse Statique de Code (SAST)

Bandit, initialement développé par OpenStack Security, est l'outil SAST (Static Application Security Testing) de référence pour Python. Il ne lit pas le code comme un simple fichier texte, mais le compile en un Arbre Syntaxique Abstrait (AST) avant de l'analyser.

Plugin ID	Catégorie ciblée	Vulnérabilités détectées	Gravité moyenne
B104	Hardcoded Bind	Bind réseau sur 0.0.0.0 (exposition globale)	Moyenne
B105/B106	Hardcoded Passwords	Mots de passe en clair dans le code	Critique
B301	Pickle	Désérialisation unsafe via module pickle (RCE)	Haute
B322	Input	Utilisation de input() en Python 2 (Injection)	Haute
B501	SSL Validation	requests.get(verify=False) (MITM)	Haute
B602	Subprocess Shell	subprocess.call(shell=True) (Command Inj.)	Critique
B608	SQL Injection	Concaténation de strings dans cursor.execute()	Critique

✓ **BONNE PRATIQUE** : Dans SecurAx, Bandit est exécuté en tant que processus enfant isolé via subprocess.Popen(). Les résultats JSON sont ensuite ingérés par ARIA pour corréler ces découvertes avec le framework MITRE ATT&CK;

3.5 python-nmap 0.7.1 — Cartographie Réseau

Nmap (Network Mapper) est l'outil d'audit réseau le plus célèbre au monde. `python-nmap` n'est pas une réécriture de Nmap, mais un wrapper qui exécute le binaire système de Nmap et parse sa sortie XML en dictionnaires Python facilement manipulables.

```
# scanners/netscan_scanner.py - Wrapper Nmap Sécurisé
import nmap
import ipaddress

def run_network_scan(target_ip):
    # 1. Validation STRICTE de la cible (Anti-Command Injection)
    try:
        ip = ipaddress.ip_address(target_ip)
    except ValueError:
        raise ValueError("Cible invalide. IP requise.")

    # 2. Initialisation du port scanner
    nm = nmap.PortScanner()
```

```
# 3. Exécution avec arguments spécifiques :
# -sV : Détection de version des services
# -Pn : Pas de ping (utile si serveur bloque ICMP)
# --script vuln : Exécute la catégorie de scripts NSE de vulnérabilité
# -T4 : Timing agressif (rapide)
nm.scan(hosts=str(ip), arguments='-sV -Pn --script vuln -T4')

return nm.all_hosts()
```

3.6 Flask-Limiter 4.1.1 — Protection Anti-Bruteforce

La prévention des attaques de déni de service (DoS) et de Bruteforce (OWASP A07) est assurée par Flask-Limiter. Il implémente l'algorithme 'Token Bucket' ou 'Leaky Bucket' pour restreindre le nombre de requêtes par IP/Session.

Endpoint / Blueprint	Limite appliquée	Menace atténuée
Application globale	1000/jour; 200/heure	DoS généralisé, Scraping abusif
/api/auth/login	5/minute; 20/heure	Bruteforce de mots de passe, Credential Stuffing
/api/auth/setup-totp	3/minute	MFA Fatigue Attack (Spamming de tokens)
/api/scans/start	1/minute par IP	Épuisement des ressources (scans Nmap coûteux en CPU)

4. ANALYSE FORENSIQUE : VULNÉRABILITÉS ET REMÉDIATION

Ce chapitre constitue le cœur technique de SecurAx. Il documente de manière exhaustive les vulnérabilités traitées par la plateforme. L'objectif n'est pas seulement de lister des failles théoriques, mais de prouver leur exploitabilité par des PoC (Proof of Concept) et de fournir les remédiations implémentées dans le code source.

4.1 Vue d'Ensemble — Matrice de Vulnérabilités

ID	Vulnérabilité (CWE)	Score CVSS	Vecteur	Impact principal
V-001	SECRET_KEY Hardcodée (CWE-798)	9.8 CRITIQUE	Lecture code/Git	Session Hijacking global
V-002	Désérialisation Insecure (CWE-502)	9.8 CRITIQUE	Payload Pickle	Remote Code Execution (RCE)
V-003	SSRF sur API NVD (CWE-918)	8.6 HAUTE	Param URL	Accès réseau interne AWS
V-004	Injection SQL (CWE-89)	8.5 HAUTE	Form. Login	Exfiltration de base de données
V-005	Path Traversal ZIP (CWE-22)	7.5 HAUTE	Upload File	Écrasement fichiers système
V-006	Reflected XSS (CWE-79)	6.1 MOYENNE	Param GET	Vol de tokens CSRF/Cookies
V-007	Timing Attack Auth (CWE-204)	5.3 MOYENNE	Auth API	Énumération d'utilisateurs
V-008	Absence de Rate Limiting (CWE-799)	5.3 MOYENNE	Network	Déni de service / Bruteforce

4.2 V-001 : SECRET_KEY Hardcodée (CWE-798)

■ **CRITIQUE : Score CVSS: 9.8 (Réseau, Complexité Faible, Sans Privilèges, Sans Interaction, Impact Total)**

Dans Flask, la `SECRET_KEY` est le cœur cryptographique de l'application. Elle est utilisée pour signer cryptographiquement les cookies de session via `itsdangerous`. Si un attaquant connaît cette clé, il peut forger n'importe quel cookie de session sans mot de passe.

Le problème (Code Vulnérable) :

```
# backend/config.py (VULNÉRABLE)
class Config:
    # La clé est stockée en texte clair dans le dépôt Git
    SECRET_KEY = 'super_secret_key_123_do_not_share'
```

Proof of Concept (PoC) d'exploitation :

Un attaquant trouve le code sur GitHub. Il peut alors utiliser le module `flask-unsign` pour générer une session d'administrateur valide :

```
$ flask-unsign --sign --cookie '{"user_id': 1, 'role': 'admin'}" --secret "super_secret_key_123_do_not_share"
eyd1c2VyX2lkJzZogMSwgJ3JvbGUnOiAnYWRTaW4nfQ.ZF1... (Cookie admin valide généré localement)
```

Remédiation implémentée dans SecurAx :

```
# backend/config.py (SÉCURISÉ)
import os

class Config:
    # La clé est chargée DEPUIS L'ENVIRONNEMENT, jamais depuis le code
    SECRET_KEY = os.environ.get('FLASK_SECRET_KEY')

    def __init__(self):
        if not self.SECRET_KEY:
            # Fail-fast : L'application refuse de démarrer si la clé manque
            raise ValueError("CRITICAL: FLASK_SECRET_KEY environment variable is not set")
```

4.3 V-004 : Injection SQL (CWE-89)

L'injection SQL survient lorsque des données contrôlées par l'utilisateur sont concaténées directement dans une requête SQL sans paramétrisation. Cela permet de modifier la logique de la requête.

Le problème (Code Vulnérable) :

```
# backend/database.py (VULNÉRABLE)
def check_user(username, password):
    # CONCATÉNATION DE CHAÎNES : Extrêmement dangereux
    query = f"SELECT * FROM users WHERE username='{username}' AND password='{password}'"
    cursor.execute(query)
```

Payloads d'exploitation testés par le Scanner DAST de SecurAx :

Payload	Type	Résultat d'exécution
' OR 1=1 --	Authentication Bypass	SELECT * FROM users WHERE username="" OR 1=1 --' AND... (Retourne le 1er utilisateur, souvent admin)
'; DROP TABLE users --	Destruction	Ferme la requête, exécute le DROP TABLE

Payload	Type	Résultat d'exécution
' UNION SELECT id, password FROM users--	Exfiltration	Combine le résultat normal avec l'extraction des hashes

Remédiation implémentée (Requêtes Préparées) :

```
# backend/models.py (SÉCURISÉ via SQLAlchemy ORM)
from .extensions import db

def get_user_by_username(username):
    # SQLAlchemy utilise des requêtes préparées (Prepared Statements) par défaut.
    # Le driver DB sépare strictement la syntaxe SQL (la logique) des données.
    return User.query.filter_by(username=username).first()
```

4.4 V-007 : Énumération par Timing Attack (CWE-204)

Souvent ignorée, cette vulnérabilité permet à un attaquant de savoir si un compte existe même si le message d'erreur est générique ('Identifiants incorrects'). Le problème réside dans le temps de réponse du serveur.

L'analyse du vecteur d'attaque :

1	Étape 1	L'attaquant tente de se connecter avec `existe_pas@email.com`.
2	Étape 2	Le serveur cherche dans la BDD. Résultat : None. Il retourne l'erreur immédiatement. (Temps: 15ms)
3	Étape 3	L'attaquant tente `vrai_admin@email.com` avec un mauvais mot de passe.
4	Étape 4	Le serveur trouve l'email. Il passe donc à l'étape de vérification du mot de passe avec `bcrypt.checkpw()`. Or, bcrypt est délibérément lent. (Temps: 415ms)
5	Conclusion	Si la requête prend > 400ms, l'attaquant SAIT que l'email est valide. Il peut extraire la base d'emails complète.

Remédiation implémentée (Dummy Hash Check) :

```
# backend/blueprints/auth.py (SÉCURISÉ)
import bcrypt

# Un hash factice calculé une fois au démarrage
DUMMY_HASH = bcrypt.hashpw(b'dummy', bcrypt.gensalt(12))

def login_user(email, password):
    user = User.query.filter_by(email=email).first()
```



```
if user:
    # L'utilisateur existe : vérification normale (prend ~400ms)
    is_valid = bcrypt.checkpw(password.encode(), user.password_hash.encode())
else:
    # L'utilisateur N'EXISTE PAS :
    # On calcule le hash factice pour SIMULER le temps de calcul
    # de bcrypt. Ainsi, la requête prendra AUSSI ~400ms.
    bcrypt.checkpw(password.encode(), DUMMY_HASH)
    is_valid = False

if is_valid:
    return "Succès"
return "Identifiants incorrects"
```


5. MOTEUR DE RISQUE — CVSS v3.1 ET INTELLIGENCE

L'un des plus grands défis de la cybersécurité moderne n'est pas de trouver des vulnérabilités, mais de savoir lesquelles corriger en premier. Le moteur de risque de SecurAx (v3) va au-delà du simple score CVSS en intégrant le contexte d'exécution et la threat intelligence en temps réel.

5.1 Limites du CVSS Brut et Apport de SecurAx

Le CVSS (Common Vulnerability Scoring System) v3.1 est le standard de l'industrie, mais il est conçu pour évaluer la *sévérité technique* d'une faille dans le vide, pas le *risque réel* pour une entreprise spécifique.

Problématique CVSS	Solution Algorithmique SecurAx v3
Ignore l'exploitation active (CISA KEV)	Multiplicateur x2.5 si la CVE est dans le catalogue CISA KEV (Known Exploited Vulnerabilities).
Biais d'inflation (Tout est 'High')	Décroissance géométrique lors de l'agrégation de multiples failles (0.85^n).
Ignore le contexte métier	Facteurs environnementaux : x1.2 si face à Internet, x1.15 si données PII présentes.
Incapacité à lier les failles	Génération automatique d'Attack Chains (corrélation de failles faibles devenant critiques).

5.2 Algorithme de Calcul du Risque (risk_engine.py)

```
# backend/risk_engine.py - Agrégation algorithmique du risque
import math

def calculate_risk_v2(findings: list[dict], metadata: dict) -> float:
    if not findings:
        return 0.0

    individual_scores = []
    for f in findings:
        base = float(f.get('cvss_score', 5.0))

        # 1. Booster par type (Ex: RCE est plus dangereux qu'un XSS à score égal)
        type_boost = get_type_booster(f.get('vuln_type', ''))

        # 2. Facteur d'exploitation réelle (CISA KEV Catalog)
        kev_factor = 2.50 if f.get('cve_id') in KEV_CATALOG else 1.0
```

```

        individual = min(10.0, base * type_boost * kev_factor)
        individual_scores.append(individual)

# 3. Agrégation géométrique (évite qu'une somme de 10 failles Info ne donne 10.0)
# La faille la plus grave compte à 100%, la 2ème à 85%, la 3ème à 72%...
sorted_scores = sorted(individual_scores, reverse=True)
raw = sum(s * (0.85 ** i) for i, s in enumerate(sorted_scores))

# 4. Normalisation Sigmoidale (borne strictement entre 0 et 10)
normalized = 10.0 * (1.0 - math.exp(-raw / 15.0))

# 5. Facteurs d'environnement (Contextualisation)
env_factor = 1.0
if metadata.get('internet_facing'): env_factor *= 1.20
if metadata.get('has_pii'):         env_factor *= 1.15

return min(10.0, normalized * env_factor)

```


6. AGENT IA — ARIA (Autonomous Risk Intelligence Agent)

ARIA est le cerveau analytique de SecurAx. Plutôt que de fournir une simple liste de failles issues d'un scanner, ARIA agit comme un analyste SOC virtuel, triant, contextualisant et proposant des remédiations spécifiques au code.

6.1 Architecture de Fallback en 5 Niveaux

Pour garantir la disponibilité du service même dans des environnements isolés (air-gapped) ou en cas de panne d'API externe, ARIA utilise un mécanisme de fallback robuste :

Nive au	Moteur	Condition d'utilisation	Caractéristiques
1	Gemini 2.5 Flash Lite	Clé API valide + Internet	Idéal. Rapide, précis, faible coût token.
2	Gemini 2.0 Flash	Si Flash Lite est indisponible	Excellent raisonnement, plus coûteux.
3	Ollama (Llama 3 / Mistral)	Serveur Ollama local configuré	100% privé, traitement en local sur GPU/CPU.
4	HuggingFace API	Si Ollama non dispo	Cloud open-source, nécessite un token gratuit.
5	Règles Heuristiques Locales	Aucune connexion ni IA locale	Base de données interne de regex et templates prédéfinis.

6.2 Ingénierie de Prompts Sécurisés (System Prompt)

Les LLMs sont sujets aux hallucinations (inventer des failles qui n'existent pas). Pour y pallier, ARIA utilise un System Prompt très restrictif ('temperature' très basse et instructions directives strictes).

```
# backend/ai_agent.py - System Prompt d'ARIA
ARIA_SYSTEM_PROMPT = """
You are ARIA, a strict Senior Cybersecurity Analyst.
Your task is to analyze the provided scan findings and output a JSON response.

CRITICAL RULES:
1. NO HALLUCINATIONS: Do not invent CVEs. If a CVE is not provided, do not guess it.
2. FORMAT: You MUST return valid JSON matching the exact schema provided.
3. ACTIONABLE: Remediation code must be specific to the language identified.
4. MITRE MAPPING: Map every finding to a valid MITRE ATT&CK T-Code (e.g., T1190).
"""
```

```
# Configuration de génération stricte
generation_config = {
    'temperature': 0.1, # Très bas pour réduire la créativité / hallucinations
    'top_p': 0.8,
    'response_mime_type': 'application/json' # Force la sortie JSON structurée
}
```


7. MODULES DE SCAN — ORCHESTRATION ET INTÉGRATION

SecurAx ne réinvente pas la roue concernant l'extraction brute de données de vulnérabilité. Il se comporte comme un orchestrateur asynchrone pour les outils open-source de pointe.

Type de Scan	Outil Sous-Jacent	Méthode d'Intégration	Cible Principale
SAST	Bandit (Py) / Semgrep	Subprocess execution + JSON Parse	Analyse de code source hors ligne
SCA (Dépendances)	pip-audit / OSV.dev	API HTTP + requirements.txt	Vulnérabilités de la supply chain
DAST	OWASP ZAP API	REST API (Spidering + Active Scan)	Applications web en cours de fonctionnement
Réseau Interne	Nmap + scripts NSE	Wrapper python-nmap	Ports, services et versions
Audit Configuration	Custom Parser (SecurAx)	Expression régulières complexes	Fichiers httpd.conf (Apache) et nginx.conf

7.1 OWASP ZAP (DAST) : Intégration via API REST

ZAP est piloté de manière asynchrone (Background Task) car un scan actif peut durer plusieurs minutes voire plusieurs heures selon la taille du site cible.

1	1. Spidering	ZAP explore récursivement l'application (crawling) pour cartographier toutes les URLs, formulaires et paramètres disponibles.
2	2. Active Scanning	ZAP injecte des payloads malveillants connus (XSS, SQLi, LFI) dans tous les points d'entrée découverts à l'étape 1.
3	3. Polling Status	Le backend SecurAx interroge l'API ZAP toutes les 5 secondes (GET /JSON/ascan/view/status) pour mettre à jour la barre de progression UI.
4	4. Parsing & Ingestion	Une fois à 100%, SecurAx récupère les alertes, les normalise dans son format unifié, et les transmet au Risk Engine.

■ **CRITIQUE** : Un scan DAST Actif injecte des données réelles (Payloads SQL, Scripts JS). Il ne doit JAMAIS être exécuté sur une base de données de production non sauvegardée, sous peine d'altérer ou supprimer des données métiers (Drop Table, modification d'identifiants).

8. NORMES ET STANDARDS DE SÉCURITÉ

L'implémentation de la sécurité ne peut se baser uniquement sur des intuitions. SecurAx s'aligne rigoureusement sur les standards internationaux reconnus.

8.1 OWASP Top 10 (2021) et Prévisions 2025

Catégorie OWASP	Risque Principal	Couverture SecurAx
A01: Broken Access Control	Élévation de privilèges	Système RBAC strict, validation de session.
A02: Cryptographic Failures	Fuite de données en clair	Bcrypt (work_factor 12), TLS/SSL scanning.
A03: Injection (SQL/NoSQL/OS)	Vol de BDD, RCE	Paramétrisation stricte, Bandit SAST plugin B608.
A05: Security Misconfiguration	Exposition de services	Scan Apache/Nginx (ServerTokens, TLS protocols).
A06: Vulnerable Components	Dépendances obsolètes	Analyse SCA avec OSV.dev (Supply chain attack).
A07: Auth Failures	Bruteforce, MFA Bypass	PyOTP, Flask-Limiter, Verrouillage compte.

8.2 Conformité Réglementaire (GDPR, PCI-DSS)

Norme / Loi	Exigence clé	Impact sur SecurAx	Statut d'implémentation
GDPR Art. 32	Sécurité par défaut (Chiffrement)	Obligation de chiffrer les exports PDF sensibles	Compliant
PCI-DSS v4	Scanner trimestriel (11.3.1)	Les rapports SecurAx servent de preuve d'audit	Compliant (SAQ-A)
NIST 800-63B	Politique de mots de passe	Règles strictes (Maj/Min/Chiffre) + 12 caractères min	Compliant

9. PENSEURS ET THÉORIES FONDAMENTALES

"La sécurité est un processus, pas un produit."

— Bruce Schneier (Cryptographe)

Cette citation fondatrice de Schneier souligne pourquoi SecurAx intègre l'historique des scans et encourage des audits continus (via CI/CD) plutôt qu'un scan ponctuel.

"Assume Breach (Supposez que vous êtes déjà compromis)."

— Modèle Zero Trust (John Kindervag)

Plutôt que de faire confiance aveuglément au réseau interne, SecurAx vérifie chaque token d'API, limite le rate-limiting même pour les utilisateurs authentifiés, et loggue l'audit de toutes les actions.

"Le facteur humain est le maillon le plus faible."

— Kevin Mitnick (Ingénierie Sociale)

Une infrastructure parfaite tombe si un administrateur se fait voler son mot de passe. C'est pourquoi SecurAx force l'utilisation du MFA (TOTP) pour tous les comptes à hauts privilèges.

10. INCIDENTS RÉELS ET ÉTUDES DE CAS FORENSIQUES

Cas historique (Année)	Vulnérabilité (Cause Racine)	Impact financier/Réputation	Comment SecurAx l'aurait prévenu
Equifax (2017)	CVE-2017-5638 (Apache Struts RCE). Composant non mis à jour.	147M victimes, 700M\$ d'amende.	Le scan de dépendances (SCA) aurait alerté sur la librairie obsolète immédiatement.
Capital One (2019)	SSRF via un WAF (ModSecurity) mal configuré + AWS Metadata.	100M clients exposés, 80M\$ d'amende.	Le scanner DAST de SecurAx teste explicitement les vecteurs SSRF (169.254.x.x).
SolarWinds (2020)	Supply Chain Attack (Backdoor SUNBURST intégrée au code source).	18000 clients, Agences Gouv US infiltrées.	L'analyse SAST poussée (Bandit) aurait pu détecter l'exfiltration de données anormale dans le code source.
Twitter/X (2020)	Ingénierie sociale menant à un bypass de l'interface d'administration.	Comptes de VIPs compromis (Obama, Musk).	MFA (TOTP) strict requis pour les actions destructrices dans le panel Admin SecurAx.

11. STRATÉGIES DE MONÉTISATION ET BUSINESS PLAN

11.1 Scénarios de Déploiement Commerciaux

1	SaaS B2B (Modèle Abonnement)	Hébergement Cloud (AWS/GCP), facturation par nombre de cibles (IPs/URLs) scannées par mois. Cible : PME n'ayant pas de budget pour un Red Team interne. (Ex: 99€/mois).
2	On-Premise (Licence Annuelle)	Déploiement sur serveur isolé de l'entreprise (Docker/Kubernetes). Nécessaire pour les banques et secteurs gouvernementaux. Facturation élevée (Ex: 15,000€/an avec support).
3	Services de Pentest Assisté	Vente de prestations intellectuelles (Consulting). SecurAx est utilisé par le consultant pour diviser le temps d'audit par 4. Facturation à la mission.
4	Plugin CI/CD (DevSecOps)	Vente d'intégration GitHub Actions / GitLab CI pour bloquer les déploiements contenant des failles critiques (Shift-Left security).

11.2 Projections de Coûts d'Infrastructure (SaaS)

Composant	Technologie	Coût Estimé Mensuel (1000 users)
Calcul (Scanners I/O)	DigitalOcean Droplets / AWS EC2	150\$ - 300\$ (Scale Horizontal)
Base de données	Supabase (PostgreSQL Managé)	25\$ (Pro Plan)
Moteur IA	Gemini 2.5 Flash API	~50\$ (Pay per Token)
Stockage (Rapports PDF)	AWS S3 / R2	10\$ (Stockage froid)
Total Mensuel	Infrastructure complète	~335\$ (Marge brute très élevée)

12. CONCLUSION ET PERSPECTIVES D'ÉVOLUTION

Ce projet de fin d'études démontre qu'il est possible de concevoir une plateforme de sécurité qui soit à la fois robuste, intelligente (grâce à l'intégration d'ARIA et du MITRE ATT&CK;), et accessible financièrement ou techniquement.

12.1 Feuille de route (Roadmap V3.0)

Plusieurs évolutions sont envisagées pour transformer ce MVP avancé en produit Enterprise :

Fonctionnalité	Impact Métier	Complexité Technique
Scanner Mobile (APK/IPA)	Couverture applicative complète (MobSF integration)	Très Haute
Intégration Active Directory (SSO)	Indispensable pour la vente aux grands comptes (SAML/OIDC)	Moyenne
Remédiation Automatique (Git PR)	ARIA crée directement une Pull Request GitHub pour corriger le code	Haute
Moteur de Threat Intelligence (CTI)	Flux en temps réel IoC (Indicators of Compromise) via MISP	Moyenne

Annexe A : Références et Standards

1	OWASP	Open Web Application Security Project - Top 10 (2021)
2	MITRE	ATT&CK; Framework (Adversarial Tactics, Techniques, and Common Knowledge)
3	NIST	National Institute of Standards and Technology - SP 800-53 (Security Controls)
4	FIRST	Forum of Incident Response and Security Teams - Définition du standard CVSS v3.1
5	CISA	Cybersecurity and Infrastructure Security Agency - KEV Catalog

✓ **BONNE PRATIQUE** : Rapport généré automatiquement par SecurAx Report Engine V3. Fin du document.